TaskFlow: Performance and Accessibility Optimization Report

Project objective: improve loading efficiency, semantic structure, keyboard access, and assistive-technology feedback using lightweight HTML, CSS, and JavaScript.

1. Project overview

Deliverables include index.html, styles.css, script.js, this report in PDF and Markdown, and a README. The page is a responsive task manager with add, complete, delete, search, filter, progress, and theme controls.

2. Baseline review

Area	Risk	Improvement
Loading	Large frameworks and remote assets	Plain JS/CSS, system fonts, no third-party requests
Rendering	Blocking script execution	JavaScript loaded with defer
Structure	Generic containers and weak landmarks	Semantic header, main, section, article, form, footer, headings and list
Keyboard	No quick path to main content	Skip link and visible focus styles
Forms	Errors unclear to screen readers	Labels, helper text, aria-describedby, aria-invalid and alert region
Dynamic UI	Updates may go unnoticed	Polite live region, accessible progress bar and pressed filter states
Responsive/motion	Small-screen and motion barriers	Responsive breakpoints, reduced-motion and forced-colors support

3. Performance improvements

- No framework or third-party runtime: avoids a large dependency bundle and extra requests.
- Deferred JavaScript: script.js uses the defer attribute so HTML parsing can continue.
- System fonts and CSS-drawn decoration: no custom font or decorative image download is needed.
- Reusable CSS custom properties and simple selectors reduce duplicated styling and keep rules maintainable.
- Rendering updates the task list and summary values in a single routine, with no polling or network calls.
- For production, minify HTML/CSS/JS and enable Brotli/gzip plus suitable cache headers at the host/CDN. Source files remain readable for assessment.

4. Accessibility improvements

- Document language and viewport metadata are declared.
- Semantic landmarks and a logical heading hierarchy help screen-reader navigation.
- A skip link and :focus-visible outlines support keyboard users.
- Form labels, helper text, aria-describedby, aria-invalid, and role=alert make validation clearer.
- A polite live region announces result-summary changes; filter controls expose aria-pressed state.
- The progress indicator exposes minimum, maximum, and current values.
- Checkboxes and delete buttons have accessible names containing each task title.
- Text labels communicate status in addition to color. Reduced-motion and forced-colors media queries are included.
- There are no images in this version, so no alt text is required. Future informative images should use contextual alt text; decorative images should use alt="".

5. Testing plan

Important: Lighthouse, WebPageTest, WAVE, and axe were not run while generating this project. No measured scores are claimed. Run the tests below on the submitted or deployed version and add the actual results if required.

- **Performance:** Chrome DevTools → Lighthouse, test mobile and desktop; record Performance, Accessibility, Best Practices, and SEO scores.
- DevTools → Network: reload with cache disabled and inspect requests, transferred bytes, and render-blocking resources.
- Optional: WebPageTest on the deployed URL; record LCP, CLS, TBT/INP where available, and total bytes.
- **Accessibility:** use Tab, Shift+Tab, Enter, and Space through all controls; verify visible focus and keyboard operation.
- Submit blank and duplicate tasks; confirm the error is announced and focus returns to the input.
- Run WAVE and axe DevTools, then manually review each finding. Automated checks do not prove full WCAG conformance.
- Test 200% zoom, a narrow viewport, reduced motion, forced colors, and if possible NVDA or VoiceOver.

6. Limitations

Tasks reset on refresh because they are stored in memory only. This is a code-level improvement and testing plan, not an independent browser audit or a formal WCAG conformance claim. Automated tools cannot identify every usability issue.

7. Learning outcomes

Performance work is not limited to minification: avoiding unnecessary dependencies, reducing requests, deferring scripts, and maintaining compact CSS can help. Accessibility is built into page structure and behavior through semantic landmarks, clear labels, live feedback, keyboard access, contrast-aware styling, and motion preferences.

8. Submission checklist

- ■ Open index.html and confirm styles.css and script.js load.
- ■ Test add, complete, delete, filter, search, and clear-completed behavior.
- ■ Use only the keyboard to reach and operate every control.
- ■ Verify empty and duplicate submissions show useful errors.
- ■ Run actual Lighthouse and WAVE/axe checks before submission if required by the instructor.